Régression par processus gaussiens pour l'émulation bayésienne de modèles incertains

Cardamone Hadrien, Yilmaz Thomas, Moncomble Hugo, Bayssiere Alexandre
Département de Mathématiques (CentraleSupélec) - Projet ST5 *Scalable GPs*
Encadrant : Julien Bect

21 janvier 2026

Résumé

Ce projet étudie la régression par processus gaussiens (GP) dans le contexte de simulateurs stochastiques, en mettant l'accent sur les approches numériques *scalables*. Nous décrivons les fondements mathématiques des GP, les contraintes de complexité associées, et les méthodes efficaces (Cholesky incomplète, Woodbury, etc.) pour traiter de grands jeux de données. Deux cas réels issus de la microbiologie alimentaire et de l'épidémiologie animale seront utilisés à terme pour valider les méthodes développées.

1. Introduction

1.1. Contexte

Ce projet porte sur l'émulation de simulateurs stochastiques, qui sont des modèles numériques conçus pour représenter le comportement de systèmes complexes comportant des sources d'incertitude ou d'aléa. De tels simulateurs apparaissent dans de nombreux domaines scientifiques et industriels où les phénomènes étudiés sont trop complexes pour être décrits par des équations analytiques explicites. Ils permettent d'explorer des scénarios variés et d'évaluer les effets de différentes configurations d'entrée sur les résultats.

Plus concrètement, la motivation finale de ce projet est d'émuler deux simulateurs stochastiques : l'un modélisant les risques microbiologiques associés à la consommation de camembert au lait cru, et l'autre simulant la propagation de la brucellose chez les bouquetins du massif du Bargy. Toutefois, le cadre du projet repose initialement sur un jeu de données adapté à l'entraînement et à l'évaluation de notre modèle, avant toute application à ces simulateurs complexes.

D'un point de vue mathématique, le problème consiste à approximer la sortie d'un simulateur stochastique, c'est-à-dire une variable aléatoire dépendant d'entrées déterministes, par un modèle génératif statistique, basé sur l'apprentissage supervisé.

Nous avons choisi d'aborder cette problématique à travers la régression par processus gaussiens (GP), une méthode bayésienne et non paramétrique. Une approche bayésienne consiste à modéliser l'incertitude sur la fonction cible à l'aide d'une distribution a priori, puis à la mettre à jour avec les données observées. Cela permet d'obtenir non seulement une prédiction, mais aussi une incertitude associée.

Le caractère non paramétrique des processus gaussiens signifie que le modèle ne repose pas sur une forme fonctionnelle fixée à l'avance (comme une droite ou un polynôme), mais qu'il ajuste sa complexité aux données disponibles. Cela en fait un outil particulièrement adapté pour modéliser des fonctions aléatoires complexes issues de simulateurs stochastiques.

1.2. Objectif et positionnement

Ce projet s'inscrit dans la continuité des recherches sur l'émulation bayésienne de simulateurs stochastiques à l'aide de processus gaussiens, en particulier dans un contexte de données massives. Plusieurs travaux ont proposé des méthodes efficaces pour répondre au défi du passage à l'échelle, en allégeant le coût computationnel associé aux modèles gaussiens classiques.

Dans ce cadre, une revue approfondie des méthodes *scalables* pour les processus gaussiens a été proposée par Liu et al. [3], qui distinguent deux grandes classes d'approches : les méthodes *globales*, qui cherchent à alléger la complexité à l'échelle de l'ensemble des données, et les méthodes *locales*, qui décomposent le problème en sous-modèles spécialisés. Cette classification met en lumière les nombreuses stratégies développées pour contourner les limitations en $O(n^3)$ des GPs classiques. Notre travail s'inscrit dans la première catégorie, en se focalisant sur des approximations de rang faible de la matrice de covariance, et en s'appuyant sur les propriétés structurelles des noyaux utilisés pour construire des méthodes à la fois précises et

computationnellement efficaces.

Dans cette perspective, Bach et Jordan [1] ont introduit une décomposition prédictive à faible rang pour les méthodes à noyaux, permettant une réduction significative de la complexité tout en conservant de bonnes performances prédictives. De son côté, Romary [5] s'est intéressé à l'utilisation de la factorisation de Cholesky incomplète comme méthode légère et précise pour le traitement de grands ensembles de données. Enfin, l'ouvrage de Suzuki [6] propose une synthèse rigoureuse des méthodes à noyaux appliquées au machine learning, en mettant l'accent sur leur formalisation mathématique et leur implémentation en Python.

Par ailleurs, les fondements algébriques nécessaires à la manipulation de grandes matrices telles que les factorisations, les techniques de réduction de rang ou l'utilisation de l'identité de Woodbury sont traités de manière approfondie dans l'ouvrage de référence de Golub et Van Loan [2], qui constitue un socle théorique essentiel pour les aspects numériques de ce travail.

Ce projet s'appuie sur cette littérature pour étudier, implémenter et comparer des techniques numériques adaptées à la régression par processus gaussiens en grande dimension. Les objets mathématiques manipulés incluent la fonction de covariance $k(\mathbf{x}, \mathbf{x}')$, la matrice de covariance $K \in \mathbb{R}^{n\times n}$, et des outils numériques tels que la factorisation de Cholesky incomplète, l'identité de Woodbury ou encore les approximations de rang faible.

Le problème mathématique peut se formuler ainsi : étant donné un ensemble de paires $(\mathbf{x}_i, y_i)$, où chaque y_i est une réalisation du simulateur en l'entrée $\mathbf{x}_i$, on cherche à approximer la loi *a posteriori* $Y(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$, tout en tenant compte des contraintes computationnelles liées à la taille de l'échantillon ($n \gg 1$).

L'objectif est double : d'une part, approfondir la compréhension théorique et algorithmique des processus gaussiens appliqués à l'émulation de simulateurs stochastiques ; d'autre part, explorer des stratégies numériques efficaces permettant de rendre ces méthodes praticables sur des ensembles de données comportant plusieurs dizaines de milliers de points. Ces approches dites *scalables* visent à surmonter les limitations computationnelles classiques de la régression gaussienne, dont la complexité mémoire est en $O(n^2)$ et le coût de calcul en $O(n^3)$.

2. Régression par processus gaussiens (GP)

Les méthodes à noyau permettent d'approximer des relations non linéaires en exploitant des fonctions de similarité. Les processus gaussiens (GP) s'inscrivent dans cette famille, et modélisent directement la distribution sur les fonctions. Un GP est entièrement défini par une moyenne $m(\mathbf{x})$ et une fonction de covariance $k(\mathbf{x}, \mathbf{x}')$.

2.1. Méthode à noyau

On cherche à approximer une fonction inconnue $f : \mathcal{X} \to \mathbb{R}$, à partir de données bruitées.

Un processus gaussien est une distribution sur un espace de fonctions. On note :

$$f \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}')) \tag{1}$$

avec :

— $\mathcal{X} \subseteq \mathbb{R}^d$: espace d'entrée,

— $m : \mathcal{X} \to \mathbb{R}$: fonction moyenne,

— $k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$: fonction noyau (ou covariance), symétrique et définie positive.

Cela signifie que pour tout ensemble fini $\{\mathbf{x}_1, \ldots, \mathbf{x}_n\} \subset \mathcal{X}$, on a :

$$\mathbf{f} = \begin{pmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_n) \end{pmatrix} \sim \mathcal{N}(\mu, \mathbf{K}), \tag{2}$$

avec :

$$\mu = \begin{pmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_n) \end{pmatrix}, \mathbf{K} = \begin{pmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & \cdots & k(\mathbf{x}_1, \mathbf{x}_n) \\ \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & \cdots & k(\mathbf{x}_n, \mathbf{x}_n) \end{pmatrix} \in \mathbb{R}^{n\times n}.$$

2.2. Observation des données

On considère un ensemble de données bruitées :

$$\mathcal{D}_n = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \tag{3}$$

où chaque observation est composée :

— d'un vecteur d'entrée $\mathbf{x}_i = (x_i^{(1)}, \ldots, x_i^{(d)})^\top \in \mathcal{X} \subseteq \mathbb{R}^d$,

— et d'une sortie scalaire $y_i \in \mathbb{R}$ donnée par le modèle :

$$y_i = f(\mathbf{x}_i) + \epsilon_i, \quad \text{avec } \epsilon_i \sim \mathcal{N}(0, \sigma^2), \text{ i.i.d.} \tag{4}$$

L'ensemble des entrées est regroupé sous forme matricielle par :

$$\mathbf{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_n)^\top \in \mathbb{R}^{n\times d}. \tag{5}$$

où chaque ligne correspond à une observation, et chaque colonne à une dimension du vecteur d'entrée. Ainsi, $x_i^{(j)}$ désigne la valeur de la j-ième variable d'entrée pour la i-ième observation.

Avant d'entraîner un processus gaussien, il est courant de normaliser les données d'entrée afin d'assurer une échelle homogène entre les dimensions. Cette transformation présente plusieurs avantages :

— elle évite qu'une dimension ne domine les autres dans le calcul des distances,

— elle stabilise l'optimisation des hyperparamètres du noyau,

— elle améliore la condition numérique de la matrice de covariance.

Dans notre cas, chaque composante $x_i^{(j)}$ est transformée selon la règle suivante :

$$x_i^{(j)} \longmapsto \frac{x_i^{(j)} - \min_k x_k^{(j)}}{\max_k x_k^{(j)} - \min_k x_k^{(j)}} \tag{6}$$

Cette transformation, appliquée indépendamment sur chaque colonne de $\mathbf{X}$, permet de ramener chaque dimension dans l'intervalle $[0, 1]$, facilitant ainsi l'apprentissage du modèle.

2.3. Choix du noyau

Dans un processus gaussien, le noyau $k(x, x')$ joue un rôle central en définissant la notion de proximité entre les points d'entrée. Cette proximité est interprétée comme une mesure de la corrélation entre les valeurs de la fonction cible aux points x et x'. Ainsi, deux points proches selon le noyau ont tendance à induire des valeurs de fonction similaires, reflétant une hypothèse de régularité de la fonction à approximer.

Le choix du noyau dépend donc des propriétés que l'on souhaite imposer à la fonction cible [4]. Par exemple, un noyau exponentiel simple, défini par :

$$k(x, x') = \exp\left(-\alpha \|x - x'\|\right) \tag{7}$$

génère des fonctions continues mais non dérivables, capturant des variations abruptes. À l'inverse, un noyau exponentiel quadratique (ou RBF), donné par :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{\ell^2}\right) \tag{8}$$

produit des fonctions infiniment dérivables, modélisant des comportements plus lisses.

2.4. Conditionnement d'une loi normale conjointe

Soit le vecteur aléatoire

$$\mathbf{Y} = \begin{pmatrix} \mathbf{Y}_1 \\ \mathbf{Y}_2 \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{pmatrix}, \begin{pmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{pmatrix} \right) \tag{9}$$

où :

— $\mathbf{Y}_1 \in \mathbb{R}^m$, $\mathbf{Y}_2 \in \mathbb{R}^n$,

— $\boldsymbol{\mu}_1$, $\boldsymbol{\mu}_2$ sont les vecteurs de moyenne respectifs,

— $K_{11} \in \mathbb{R}^{m \times m}$, $K_{22} \in \mathbb{R}^{n \times n}$,

— $K_{12} = K_{21}^\top \in \mathbb{R}^{m \times n}$.

La formule générale de l'espérance conditionnelle d'un vecteur gaussien est :

$$\begin{aligned} \mathbb{E}[\mathbf{Y}_2 \mid \mathbf{Y}_1 = \mathbf{y}_1] &= \boldsymbol{\mu}_2 + \mathrm{Cov}(\mathbf{Y}_2, \mathbf{Y}_1)\, \mathrm{Var}(\mathbf{Y}_1)^{-1} (\mathbf{y}_1 - \boldsymbol{\mu}_1) \\ &= \boldsymbol{\mu}_2 + K_{21} K_{11}^{-1} (\mathbf{y}_1 - \boldsymbol{\mu}_1). \end{aligned}$$

La covariance conditionnelle est donnée par :

$$\begin{aligned} \mathrm{Cov}(\mathbf{Y}_2 \mid \mathbf{Y}_1) &= \mathrm{Var}(\mathbf{Y}_2) - \mathrm{Cov}(\mathbf{Y}_2, \mathbf{Y}_1)\, \mathrm{Var}(\mathbf{Y}_1)^{-1} \\ &\quad \cdot \mathrm{Cov}(\mathbf{Y}_1, \mathbf{Y}_2) \\ &= K_{22} - K_{21} K_{11}^{-1} K_{12}. \end{aligned}$$

Alors, la loi conditionnelle de $\mathbf{Y}_2$ sachant $\mathbf{Y}_1 = \mathbf{y}_1$ est une loi normale multivariée :

$$\begin{aligned} \mathbf{Y}_2 \mid \mathbf{Y}_1 = \mathbf{y}_1 \sim \mathcal{N}\big(&\boldsymbol{\mu}_2 + K_{21} K_{11}^{-1} (\mathbf{y}_1 - \boldsymbol{\mu}_1), \\ &K_{22} - K_{21} K_{11}^{-1} K_{12}\big). \end{aligned} \tag{10}$$

2.5. Distribution conditionnelle

Dans le cadre de la regression par processus gaussiens, les variables Y_1 et Y_2 correspondent respectivement aux valeurs du processus observées avec du bruit sur l'ensemble d'entraînement et aux valeurs à prédire sur de nouveaux points, ce qui permet d'exprimer explicitement la distribution conditionnelle du processus.

Soit $\mathbf{x}^* \in \mathcal{X}$ un point test. On souhaite estimer la distribution de la variable aléatoire $f(\mathbf{x}^*)$, conditionnellement aux observations.

$$f(\mathbf{x}^*) \mid \mathcal{D}_n \sim \mathcal{N}(m_n(\mathbf{x}^*), \sigma_n^2(\mathbf{x}^*)) \tag{11}$$

où la moyenne s'écrit :

$$m_n(\mathbf{x}^*) = \mathbf{k}(\mathbf{x}^*, \mathbf{X})(K_{nn} + \sigma^2 I_n)^{-1} \mathbf{y}_n, \tag{12}$$

et la variance :

$$\sigma_n^2(\mathbf{x}^*) = k(\mathbf{x}^*, \mathbf{x}^*) - \mathbf{k}(\mathbf{x}^*, \mathbf{X})(K_{nn} + \sigma^2 I_n)^{-1} \mathbf{k}(\mathbf{X}, \mathbf{x}^*). \tag{13}$$

avec :

— $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n]^\top \in \mathbb{R}^{n \times d}$,

— $\mathbf{y}_n = [y_1, \ldots, y_n]^\top \in \mathbb{R}^n$,

— $K_{nn} \in \mathbb{R}^{n \times n}$, $[K_{nn}]_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$,

— $\mathbf{k}(\mathbf{x}^*, \mathbf{X}) = [k(\mathbf{x}^*, \mathbf{x}_1), \ldots, k(\mathbf{x}^*, \mathbf{x}_n)] \in \mathbb{R}^{1 \times n}$,

— $I_n \in \mathbb{R}^{n \times n}$: matrice identité.

Dans le cadre de ce projet, et par souci de simplification, nous supposerons que $m_n(\mathbf{x}^*) = 0$, c'est-à-dire que la moyenne *a posteriori* est nulle. Cette hypothèse est classique et n'affecte pas la généralité du modèle, car la moyenne peut être réintroduite si nécessaire.

3. Amélioration computationnelle

Dans l'implémentation pratique des processus gaussiens, une étape clé est l'inversion de la matrice de covariance bruitée :

$$K_\sigma := K + \sigma^2 I, \tag{14}$$

où $K \in \mathbb{R}^{n\times n}$ est la matrice noyau évaluée sur les données d'entraînement, I est la matrice identité de dimension n, et $\sigma^2 > 0$ est la variance du bruit observationnel. Cette matrice K_σ doit être inversée dans plusieurs étapes du calcul, or, cette matrice est de taille $n \times n$, et son inversion directe, même avec des bibliothèques spécialisées, aurait un coût de $\mathcal{O}(n^3)$, prohibitif pour des jeux de données de taille modérée à grande. L'inversion directe étant trop coûteuse, on exploite une structure favorable : le caractère symétrique défini positif de K_σ, pour utiliser la décomposition de Cholesky, permettant ensuite une plus grande efficacité numérique en divisant le temps de calcul par 3.

3.1. Théorème de décomposition de Cholesky

Soit $A \in \mathbb{R}^{n\times n}$ une matrice réelle symétrique définie positive, c'est-à-dire telle que :

$$A = A^\top \quad \text{et} \quad \forall x \in \mathbb{R}^n \setminus \{0\},\ x^\top A x > 0. \tag{15}$$

Alors, il existe une matrice triangulaire inférieure $L \in \mathbb{R}^{n\times n}$ à diagonale strictement positive telle que :

$$A = LL^\top. \tag{16}$$

Cette factorisation est unique.

La matrice K_σ satisfait les hypothèses du théorème :

— K est une matrice de covariance, donc symétrique semi-définie positive ;

— l'ajout de $\sigma^2 I$ avec $\sigma^2 > 0$ la rend strictement définie positive.

3.2. Algorithme de la décomposition de Cholesky

Dans la pratique, la décomposition de Cholesky $K_\sigma = LL^\top$, dont l'existence est assurée par le théorème précédent, peut être calculée de manière efficace à l'aide d'un algorithme simple, basé sur le calcul incrémental des coefficients de la matrice triangulaire inférieure L.

L'algorithme procède ligne par ligne (ou colonne par colonne), selon les formules suivantes :

$$\begin{aligned} l_{ii} &= \sqrt{K_{\sigma,ii} - \sum_{k=1}^{i-1} l_{ik}^2}, \quad \text{pour } i = 1, \dots, n, \\ l_{ij} &= \frac{1}{l_{jj}} \left(K_{\sigma,ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right), \quad \text{pour } i = j+1, \dots, n. \end{aligned} \tag{17}$$

Autrement dit, chaque coefficient l_{ij} est obtenu à partir des termes déjà calculés de L, ce qui permet une factorisation incrémentale, sans jamais devoir manipuler l'inverse de K_σ ni recourir à des opérations coûteuses.

3.3. Avantages numériques : complexité et stabilité

Le calcul de la décomposition de Cholesky de K_σ s'effectue avec une complexité en temps de $\frac{n^3}{3}$, ce qui est significativement plus rapide que l'inversion directe de la matrice. Une fois la factorisation $K_\sigma = LL^\top$ obtenue, on peut :

— Résoudre des systèmes linéaires $K_\sigma x = b$ par substitution avant/après en résolvant successivement :

$$Ly = b, \quad L^\top x = y,$$

avec une complexité en $\mathcal{O}(n^2)$;

— Calculer l'inverse de K_σ via sa décomposition de Cholesky.

3.4. Résumé opérationnel

Une fois la décomposition de Cholesky obtenue :

$$K_\sigma = LL^\top,$$

où $L \in \mathbb{R}^{n\times n}$ est une matrice triangulaire inférieure à diagonale strictement positive, elle permet de simplifier et d'accélérer les calculs de la moyenne et de la variance prédictives.

Calcul de la moyenne prédictive. On rappelle que la moyenne prédictive s'écrit :

$$\mu_* = k_*^\top K_\sigma^{-1} y.$$

Plutôt que de calculer explicitement K_σ^{-1}, on résout deux systèmes triangulaires successifs :

$$Lz = y \quad \text{(substitution avant)}, \qquad L^\top \alpha = z \quad \text{(substitution arrière)},$$

ce qui donne directement $\alpha = K_\sigma^{-1} y$, avec un coût total en $\mathcal{O}(n^2)$ après une phase de factorisation en $\mathcal{O}(n^3/3)$.

Calcul de la variance prédictive La variance prédictive s'écrit :

$$\sigma_*^2 = k_{**} - k_*^\top K_\sigma^{-1} k_*.$$

On applique la même stratégie : au lieu de calculer $K_\sigma^{-1} k_*$, on résout :

$$Lv = k_*,$$

puis on calcule :

$$\sigma_*^2 = k_{**} - v^\top v.$$

3.5. Décomposition de Cholesky incomplète

Lorsque la taille n de la matrice de covariance $K \in \mathbb{R}^{n\times n}$ devient trop grande, la complexité en $\mathcal{O}(n^3/3)$ de la décomposition de Cholesky complète peut devenir prohibitive. Pour pallier cette difficulté, on utilise fréquemment une décomposition de Cholesky incomplète, notée $\tilde{L}$, qui constitue une approximation parcimonieuse de la

matrice L issue de la décomposition exacte.

Principe :

Plutôt que de calculer l'intégralité de la matrice L telle que $K = LL^\top$, l'idée est de ne conserver que certains coefficients significatifs de L lors de la factorisation. On obtient ainsi une matrice triangulaire inférieure $\tilde{L}$, plus creuse que L, telle que :

$$K \approx \tilde{L}\tilde{L}^\top. \tag{18}$$

Cette approximation permet des gains substantiels en temps de calcul et en mémoire, tout en maintenant une erreur de factorisation contrôlée.

Hypothèses :

Dans le cadre des processus gaussiens, la matrice K est une matrice de covariance symétrique définie positive, construite à partir d'un noyau reproduisant. Elle possède un spectre réel positif $(\lambda_1, \ldots, \lambda_n)$, qui présente souvent une décroissance rapide :

$$\lambda_1 \gg \lambda_2 \gg \cdots \gg \lambda_r \gg \lambda_{r+1} \approx \cdots \approx \lambda_n \approx 0,$$

pour un certain rang effectif $r \ll n$. Cette structure spectrale résulte à la fois de la régularité du noyau utilisé et de la redondance statistique des données.

Cette structure spectrale permet d'introduire de manière intuitive une approximation de rang faible :

1. La matrice K est proche (dans un sens que l'on précisera) d'une matrice de rang faible. Il existe une approximation à rang réduit :

$$K \approx \sum_{i=1}^{r} \lambda_i u_i u_i^\top, \tag{19}$$

   où les u_i sont les vecteurs propres associés aux plus grandes valeurs propres.
2. En conséquence, la majeure partie de l'information contenue dans la factorisation de Cholesky exacte $K = LL^\top$ est concentrée dans un nombre fini de colones L, tandis que les autres peuvent être négligés sans perte significative de précision.

Dans ce contexte, une décomposition de Cholesky incomplète permet d'obtenir efficacement une approximation de rang faible de la matrice K. En négligeant les contributions correspondant aux petites valeurs propres, on construit une matrice $\tilde{L}$ telle que :

$$K \approx \tilde{L}\tilde{L}^\top \quad \Rightarrow \quad K_\sigma = K + \sigma^2 I \approx \tilde{L}\tilde{L}^\top + \sigma^2 I. \tag{20}$$

Cette approximation permet ensuite une inversion rapide de K_σ via des techniques adaptées aux matrices de faible rang (voir section suivante), tout en garantissant un contrôle sur l'erreur :

$$\|K - \tilde{L}\tilde{L}^\top\| \leq \varepsilon, \tag{21}$$

où $\|\cdot\|$ désigne typiquement la norme spectrale ou la norme de Frobenius, et $\varepsilon > 0$ est un seuil fixé selon le critère de troncature ou le niveau de remplissage autorisé.

Formule de Woodbury : Lorsque l'on approxime la matrice de covariance K par une matrice de faible rang, nous faisons alors appel à l'identité de Woodbury, qui permet un gain d'efficacité notable dans notre cas, où la matrice à inverser s'écrit $K_\sigma = \tilde{L}\tilde{L}^\top + \sigma^2 I$, somme d'un terme de rang faible et d'un multiple de l'identité.

Théorème (identité de Woodbury) : Soient $A \in \mathbb{R}^{n\times n}$, $U \in \mathbb{R}^{n\times k}$, $C \in \mathbb{R}^{k\times k}$, et $V \in \mathbb{R}^{k\times n}$ telles que :

— A est inversible,

— C est inversible,

Alors, la matrice $A + UCV \in \mathbb{R}^{n\times n}$ est inversible, et on a :

$$(A+UCV)^{-1} = A^{-1} - A^{-1}U(C^{-1} + VA^{-1}U)^{-1}VA^{-1}.$$

Application à notre cas : Dans notre situation, nous cherchons à inverser la matrice :

$$K_\sigma = K + \sigma^2 I \approx \tilde{L}\tilde{L}^\top + \sigma^2 I. \tag{22}$$

On reconnaît ici la structure $A + UCV$ avec les identifications suivantes :

$$A = \sigma^2 I, \quad U = \tilde{L}, \quad C = I, \quad V = \tilde{L}^\top. \tag{23}$$

Les hypothèses du théorème sont bien vérifiées :

— $\sigma^2 I$ est inversible.

— $C = I$ est également inversible.

— les dimensions de $\tilde{L}$ et $\tilde{L}^\top$ sont compatibles avec A.

On peut donc appliquer l'identité de Woodbury pour approximer l'inverse de K_σ, ce qui donne :

$$\begin{aligned} K_\sigma^{-1} &\approx (\tilde{L}\tilde{L}^\top + \sigma^2 I)^{-1} \\ &= \frac{1}{\sigma^2} I - \frac{1}{\sigma^2}\tilde{L}\left(I + \frac{1}{\sigma^2}\tilde{L}^\top\tilde{L}\right)^{-1}\tilde{L}^\top\frac{1}{\sigma^2}. \end{aligned} \tag{24}$$

Cette expression permet de remplacer l'inversion d'une matrice $n \times n$ par celle d'une matrice beaucoup plus petite $r \times r$, où r est le rang de $\tilde{L}$, conduisant à une réduction significative de la complexité à $\mathcal{O}(nr^2 + r^3)$, contre $\mathcal{O}(n^3)$ en inversion directe.

3.5. Décomposition incomplète de Cholesky avec pivots (Bach & Jordan)

Dans le cadre de l'approximation des noyaux, une méthode efficace consiste à effectuer une décomposition incomplète de Cholesky avec pivot, telle que développée par Bach et Jordan [1]. Cette méthode permet de construire une

approximation de rang faible d'une matrice de covariance $K \in \mathbb{R}^{n\times n}$, symétrique définie positive, sous la forme $K \approx LL^\top$, tout en contrôlant explicitement l'erreur d'approximation et la complexité de calcul. Contrairement à une décomposition naïve, le recours aux pivots est essentiel car l'information n'est pas nécessairement concentrée dans les premières colonnes de K ; la stratégie de pivot permet ainsi de sélectionner les colonnes les plus informatives, et donc de réduire plus efficacement l'erreur à chaque étape de la factorisation.

3.5.1 Algorithme de décomposition

L'algorithme construit une matrice $L \in \mathbb{R}^{n\times m}$ colonne par colonne, où $m \ll n$, tel que $A = LL^\top$ est une approximation de K. On initialise une matrice diagonale $D \in \mathbb{R}^{n\times n}$ avec $D(i) = K_{ii}$.

Soit $I_k = \{i_1, \dots, i_k\}$ l'ensemble des indices des k pivots sélectionnés, et J_k son complément dans $\{1, \dots, n\}$ (trié). À chaque itération k, en supposant i_k déjà connu, l'étape de mise à jour est la suivante :

- $L(i_k, k) = \sqrt{D(i_k)}$
- $\forall j \in J_k, \quad L(j,k) = \dfrac{1}{L(i_k,k)} \left(K(j,i_k) - \sum_{\ell=1}^{k-1} L(j,\ell) L(i_k,\ell) \right)$
- $\forall j \in J_k, \quad D(j) \leftarrow D(j) - L(j,k)^2$

L'approximation au rang k est donnée par $A_k = L_k L_k^\top$, où L_k désigne les k premières colonnes de L.

3.5.2 Propriétés de la matrice approchée

Une propriété remarquable de cette méthode est que l'approximation $A = LL^\top$ obtenue après m étapes satisfait les conditions suivantes :

— A est symétrique définie positive et de rang m.

— l'espace colonne de A est contenu dans celui des colonnes de K indexées par $I = \{i_1, \dots, i_m\}$.

— $A(:, I) = K(:, I)$, c'est-à-dire que l'approximation conserve exactement les colonnes pivot de K.

En partitionnant K selon I et son complément J, on peut écrire l'approximation A sous forme bloc :

$$A = \begin{pmatrix} K(I,I) & K(I,J)^\top \\ K(I,J) & K(J,I)\,K(I,I)^{-1}\,K(J,I)^\top \end{pmatrix}. \quad (25)$$

On dispose également de la décomposition complète

$$K = \begin{pmatrix} K(I,I) & K(I,J)^\top \\ K(I,J) & K(J,J) \end{pmatrix}, \quad (26)$$

$$K - A = \begin{pmatrix} 0 & 0 \\ 0 & K(J,J) - K(J,I)\,K(I,I)^{-1}\,K(J,I)^\top \end{pmatrix}. \quad (27)$$

La matrice $K(I,I)$ est inversible car ses indices sont choisis parmi les pivots, et K est symétrique définie positive. Par l'inégalité de Cauchy–Schwarz (ou, de manière équivalente, par le théorème du complément de Schur), on a alors

$$K(J,J) - K(J,I)\,K(I,I)^{-1}\,K(J,I)^\top \succeq 0, \quad (28)$$

ce qui entraîne $K - A \succeq 0$. Cette positivité semi-définie du résidu légitime l'usage de la trace $\mathrm{tr}(K-A)$ comme mesure non négative et monotone de l'erreur d'approximation.

3.5.3 Choix du pivot et critère d'arrêt

La sélection du pivot i_k repose sur une stratégie gloutonne visant à maximiser l'amélioration minimale de l'approximation. On cherche à choisir le pivot qui entraîne la plus grande réduction de l'erreur $K - A_k$ mesurée en norme trace $\|K - A_k\|_1 = \mathrm{tr}(K - A_k)$. Cette norme étant coûteuse à évaluer exactement à chaque itération, on utilise un mino- rant explicite du gain apporté par chaque pivot possible.

La propriété principale de la décomposition de Cholesky incomplète est, par construction :

$$\|K - LL^\top\|_1 \le \eta \quad (29)$$

exprime une erreur absolue : on fixe une tolérance η sur l'écart entre la matrice K et son approximation $LL^\top$.

Cependant, le problème est que cette erreur η n'a pas d'échelle : elle dépend des unités ou de la taille de K, donc n'est pas comparable entre différents cas.

Pour cela, on normalise l'erreur en la rapportant à la norme de K elle-même. On introduit alors une quantité relative :

$$\|K - LL^\top\|_1 \le \eta^* \|K\|_1, \quad (30)$$

où l'on définit $\eta^* = \eta / \|K\|_1$, et donc :

$$\eta^* = \frac{\text{erreur}}{\text{valeur totale}}. \quad (31)$$

Le paramètre η^* peut alors être interprété comme la proportion de variance perdue. Autrement dit, la proportion $1 - \eta^*$ de la variance est préservée dans l'approximation.

Ainsi, après l étapes, on dispose d'une approximation de la forme :

$$\widetilde{K}_l = \widetilde{L}_l \widetilde{L}_l^\top, \quad (32)$$

où $\widetilde{L}_l$ est une matrice $n \times l$.

La norme trace de l'erreur d'approximation est donnée par :

$$\|K - \widetilde{L}_l \widetilde{L}_l^\top\|_1 = \mathrm{tr}\left(K - \widetilde{L}_l \widetilde{L}_l^\top\right) = \mathrm{tr}(K) - \sum_{i=1}^{l} \|\widetilde{L}[:, i]\|_2^2. \quad (33)$$

De fait, le critère d'arrêt de l'algorithme va être défini à chaque opération par :

$$\mathrm{tr}(K) - \sum_{i=1}^{l} \|\widetilde{L}[:,i]\|_2^2 \leq \eta^* \cdot \mathrm{tr}(K). \tag{34}$$

$$(1-\eta^*) \cdot \mathrm{tr}(K) \leq \sum_{i=1}^{l} \|\widetilde{L}[:,i]\|_2^2. \tag{35}$$

En effet, on remarque que l'ajout de la colonne $L(:,k)$ à L entraîne un gain en norme trace de $\|L(:,k)\|_2^2$, minoré par $L(i_k,k)^2 = D(i_k)$. On dispose en particulier de la décomposition.

$$\|L(:,k)\|_2^2 \;=\; L(i_k,k)^2 \;+ \sum_{j \neq i_k} L(j,k)^2, \tag{36}$$

qui met en évidence que la contribution pivot $L(i_k,k)^2$ constitue une borne inférieure immédiate du gain, accessible avant la mise à jour.

Le choix du pivot est donc effectué de manière à maximiser le gain suivant :

$$i_k = \arg\max_{i \in J_{k-1}} D(i), \tag{37}$$

ce qui garantit une amélioration minimale de l'approxi- mation. Ce critère fournit également un test d'arrêt naturel : on interrompt l'algorithme dès que $\max_{i \in J_{k-1}} D(i) < \varepsilon$, pour une tolérance donnée $\varepsilon > 0$.

3.5.4 Gain de complexité

L'algorithme de décomposition incomplète de Cholesky avec pivots présente un gain de complexité significatif par rapport à une décomposition de Cholesky complète. À chaque itération, la mise à jour de la matrice L nécessite un coût $\mathcal{O}(kn)$, où k est le nombre de pivots sélectionnés. L'algorithme complet, après m itérations, aura une complexité totale de $\mathcal{O}(m^2 n)$, ce qui est nettement plus faible que la complexité $\mathcal{O}(n^3)$ de la décomposition de Cholesky complète.

4. Résultats

Cette section évalue empiriquement l'efficacité de l'approximation de rang faible pour les matrices de covariance dans le cadre des processus gaussiens.

4.1. Données de test

Nous disposons pour ce projet d'un jeu de données connu sous le nom de *borehole*, structuré en deux fichiers distincts :

— Un fichier d'entraînement contenant 50 000 lignes, où chaque ligne correspond à un couple $(\mathbf{x}_i, y_i)$. Ici, $\mathbf{x}_i \in \mathbb{R}^7$ est un vecteur d'entrée représentant un point d'observation, et $y_i = f(\mathbf{x}_i)$, avec $f : \mathbb{R}^7 \to \mathbb{R}$ une fonction cible à approximer ;

— Un fichier de test contenant également environ 50 000 lignes, structuré de manière identique, destiné à évaluer la capacité de généralisation du modèle.

Pour construire la matrice de covariance du processus gaussien, nous utilisons un noyau à base radiale (RBF) :

$$k(x,x') = \sigma_f^2 \exp\left(-\frac{\|x-x'\|^2}{\ell^2}\right). \tag{38}$$

Dans notre cas, après une normalisation des données, nous avons choisi les hyperparamètres suivants :

$$\ell = \sqrt{2}, \quad \sigma_f^2 = 1.$$

Ces ressources seront utilisées pour évaluer empiriquement la performance de notre processus gaussien *scalable*, en particulier dans un contexte de données massives.

4.2. Utilité de l'approximation de rang faible pour les matrices de covariance

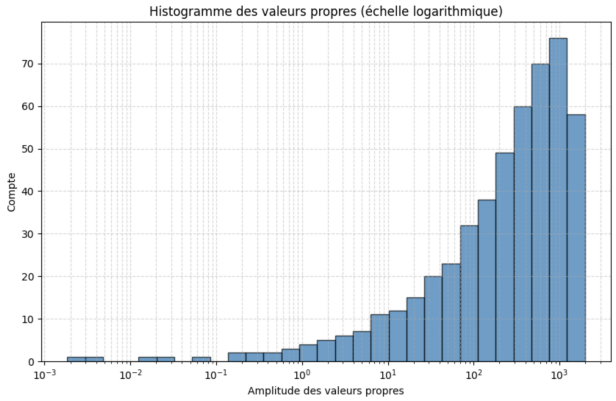


FIGURE 1 : Répartition des valeurs propres - Matrice SDP générée aléatoirement
.

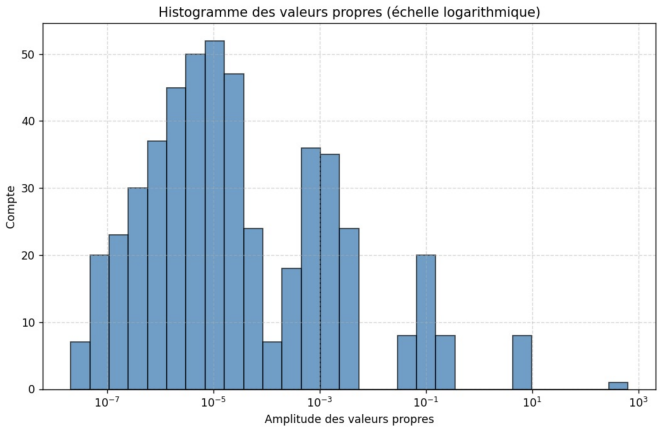


FIGURE 2 : Répartition des valeurs propres - Matrice de covariance issue du dataset *borehole*

Les figures 1 et 2 illustrent la distribution des valeurs propres (en échelle logarithmique) pour deux matrices symétriques définies positives (SDP), à travers des histogrammes. La première (Figure 1) est obtenue en générant une matrice aléatoire A de taille 500x500, dont les coefficients suivent une loi normale centrée réduite, puis en formant le produit $C = AA^\top$. La seconde (Figure 2) correspond à une matrice de covariance construite à partir d'un noyau gaussien appliqué sur un ensemble de points du dataset fourni.

Dans les deux cas, l'analyse spectrale est essentielle pour comprendre les performances des méthodes d'approximation de rang faible. Une décroissance rapide des valeurs propres permet en effet de ne conserver qu'un sous-ensemble restreint de directions spectrales significatives tout en préservant l'essentiel de l'information, ce qui se traduit par des gains significatifs en complexité computationnelle.

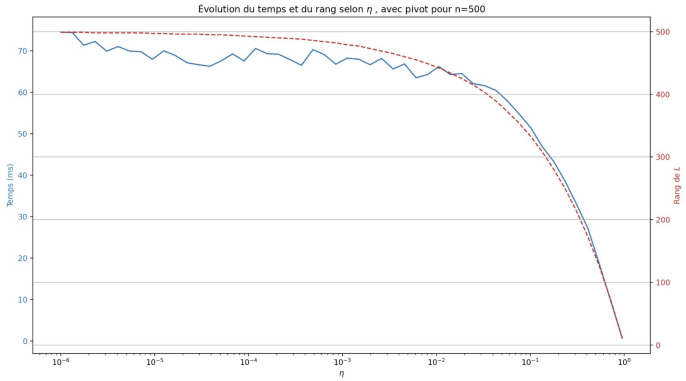


FIGURE 3 : Évolution du temps et du rang selon η - Matrice SDP générée aléatoirement.

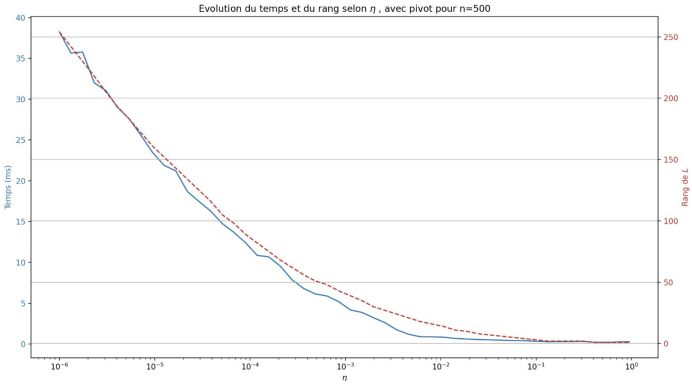


FIGURE 4 : Évolution du temps et du rang selon η - Matrice de covariance issue du dataset *borehole*.

Dans le cas de la matrice SDP générée aléatoirement Figure 3, les valeurs propres sont majoritairement concentrées à des amplitudes élevées, avec une décroissance relativement progressive. Cela implique que l'information (au sens de la trace) est répartie sur un grand nombre de directions principales. Par conséquent, il faut conserver un plus grand nombre de colonnes dans la décomposition de Cholesky incomplète avec pivot pour atteindre un seuil donné de variance expliquée, défini par le critère :

$$(1 - \eta^*) \cdot \mathrm{tr}(C) \leq \sum_i \|G[:, i]\|_2^2. \tag{39}$$

À l'inverse, la matrice de covariance Figure 4 présente un spectre fortement déséquilibré, avec une majorité de petites valeurs propres proches de zéro. Ce comportement, typique des noyaux lisses, permet une compression plus efficace : une faible proportion des directions principales porte l'essentiel de la variance. Ainsi, l'algorithme de Cholesky incomplet avec pivot atteint rapidement le seuil de variance expliquée fixé (faible η^*), ce qui justifie pleinement l'intérêt de cette méthode dans un contexte de modélisation par processus gaussiens.

4.3. Comparaison temporelle des méthodes de Cholesky

L'objectif de cette section est de comparer les performances temporelles des décompositions de Cholesky complète et incomplète, à partir de données issues d'un dataset fourni, sur lequel a été appliqué le noyau gaussien.

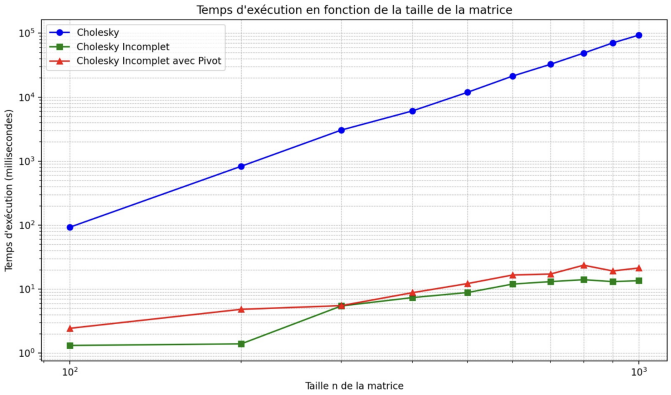


FIGURE 5 : Comparaison temporelle des méthodes de Cholesky - Matrice de covariance issue du dataset *borehole* $\eta = 0.001$.

La Figure 5 met en évidence l'efficacité remarquable de la méthode de Cholesky incomplète, avec ou sans pivot, face à la décomposition de Cholesky complète. Cette dernière montre une croissance exponentielle du temps d'exécution avec la taille de la matrice, tandis que les méthodes incomplètes conservent un temps de calcul nettement plus faible. Ce comportement confirme bien la réduction de complexité attendue avec les versions incomplètes. Une telle efficacité est particulièrement pertinente dans le cadre des processus gaussiens, où la scalabilité dépend fortement de la capacité à manipuler de grandes matrices de covariance de manière rapide et efficiente.

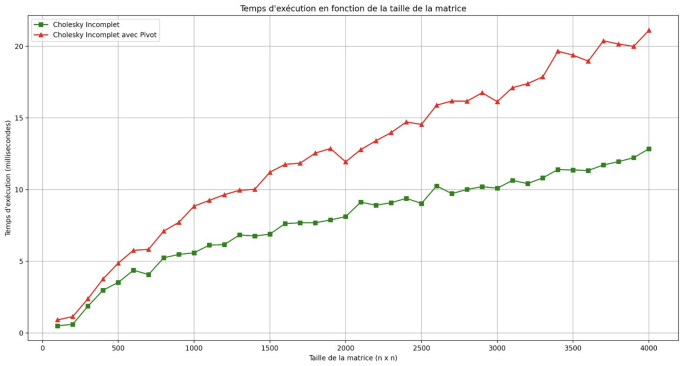


FIGURE 6 : Comparaison temporelle des méthodes de Cholesky incomplète avec et sans pivot - Matrice de covariance issue du dataset *borehole* $\eta = 0.001$.

La Figure 6 met en évidence un résultat surprenant : l'introduction du pivot dans la méthode de Cholesky incomplète entraîne systématiquement un surcoût en temps de calcul. Contrairement à ce que l'on pourrait attendre, le pivot ne permet pas de gains de performance, bien qu'il soit généralement introduit pour améliorer la stabilité numérique.

La première explication qui vient à l'esprit est liée au fonctionnement du cache processeur. Cholesky incomplet sans pivot étant un algorithme rigide, c'est-à-dire qu'il suit un parcours linéaire des colonnes de la matrice, il bénéficie davantage de la mémoire cache. Cependant, cette hypothèse a été écartée, car nous avons explicitement vidé le cache entre chaque appel à la décomposition de Cholesky durant les tests.

En réalité, la meilleure performance de Cholesky sans pivot s'explique par le comportement du rang effectif de l'approximation. À partir d'un certain seuil de taille, ce rang tend à stagner : autrement dit, les premières colonnes de la matrice de covariance suffisent à capturer les principales corrélations entre les variables aléatoires. Ainsi, faire croître la matrice n'améliore plus significativement l'approximation, et le coût computationnel marginal reste limité.

En revanche, la version avec pivot continue à explorer activement l'ensemble des colonnes, y compris les nouvelles, pour sélectionner les meilleurs candidats à chaque itération. Ce processus devient plus coûteux à mesure que la taille de la matrice augmente, d'où l'accroissement net du temps d'exécution observé.

Enfin, on peut noter que même si le rang reste stable, le temps d'exécution de la version sans pivot continue à croître, car les colonnes traitées deviennent de plus en plus longues — leur nombre ne change pas, mais leur dimension augmente, ce qui contribue naturellement à la hausse du coût de calcul.

4.4. Comparaison du rang des méthodes de Cholesky

L'objectif de cette section est de comparer le rang faible obtenu par les décompositions de Cholesky complète et incomplète (avec et sans pivot).

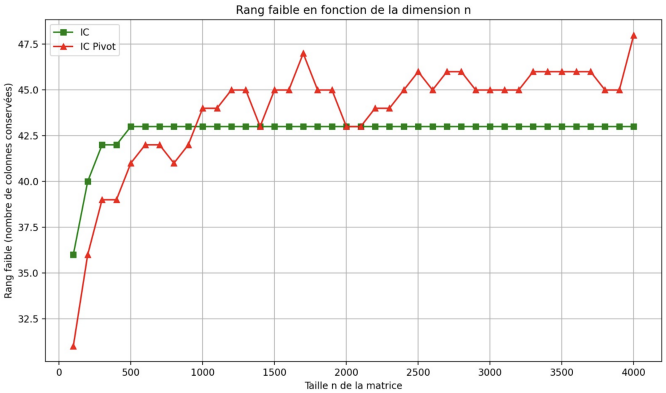


FIGURE 7 : Évolution du rang - Matrice de covariance issue du dataset *borehole* : inversion avec Cholesky incomplète $\eta = 0.001$.

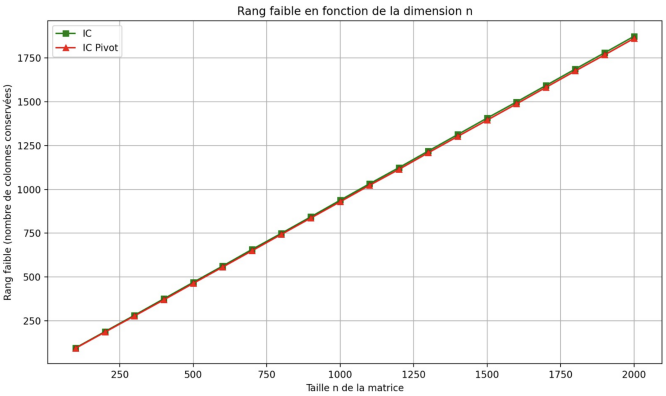


FIGURE 8 : Évolution du rang - Matrice SDP générée aléatoirement : inversion avec Cholesky incomplète $\eta = 0.001$.

Sur des données réelles, comme illustré dans la Figure 7, on observe que le rang faible obtenu par factorisation de Cholesky incomplète sans pivot stagne à partir d'une certaine taille de matrice, et devient même inférieur à celui obtenu avec pivot. Ce phénomène s'explique par la redondance présente dans les données structurées, qui limite progressivement la diversité des directions exploitées sans réordonnancement. L'utilisation du pivot permet alors de sélectionner de manière plus judicieuse les colonnes informatives, et ainsi d'augmenter le rang atteint à précision fixée.

En revanche, sur des matrices aléatoires issues d'une loi normale centrée réduite, comme illustré en Figure 8, le rang augmente régulièrement avec la taille de la matrice, et le pivot ne procure pas de gain notable. En effet, dans un contexte aléatoire, chaque vecteur est statistiquement indépendant des autres, ce qui garantit une richesse structurelle suffisante pour que l'algorithme sans pivot exploite déjà l'essentiel de l'information disponible.

4.5. Régression par processus Gaussien

Il convient de vérifier que cette amélioration computationelle ne compromet pas la qualité des prédictions. Pour cela, nous étudions la fiabilité de la régression par processus gaussien en une dimension en prenant pour exemple la reconstruction d'une fonction sinus avec un bruit suivant une loi normale centrée de variance $\sigma = 0{,}001$.

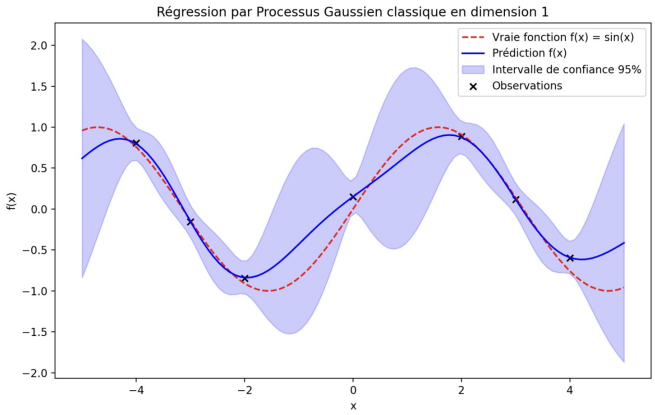


FIGURE 9 : Régression par processus gaussien de la fonction sinus : inversion avec Cholesky complète.

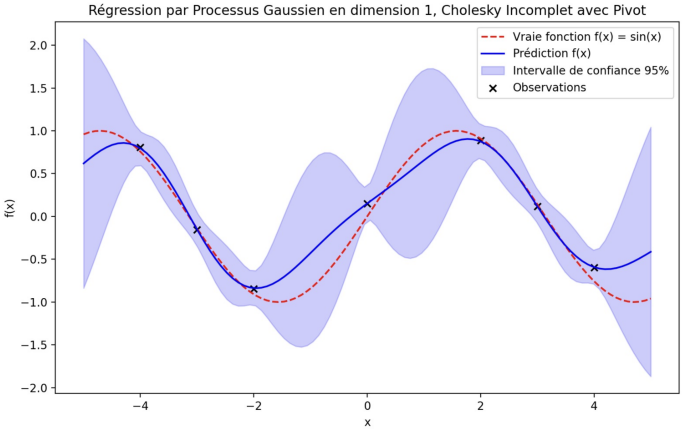


FIGURE 10 : Régression par processus gaussien de la fonction sinus : inversion par Cholesky incomplète avec pivot $\eta = 0.001$.

Les Figures 9 et 10 montrent que la réduction du temps de calcul permise par l'utilisation de la méthode de Cholesky incomplète avec pivot ne se fait pas au détriment de la précision des prédictions. La qualité de l'ajustement, ainsi que la forme et l'amplitude des intervalles de confiance, restent comparables à celles obtenues avec l'inversion classique. Ces observations confirment que l'approximation introduite ne compromet pas la fiabilité du modèle, validant ainsi l'intérêt de cette méthode dans un contexte où la scalabilité est un enjeu majeur.

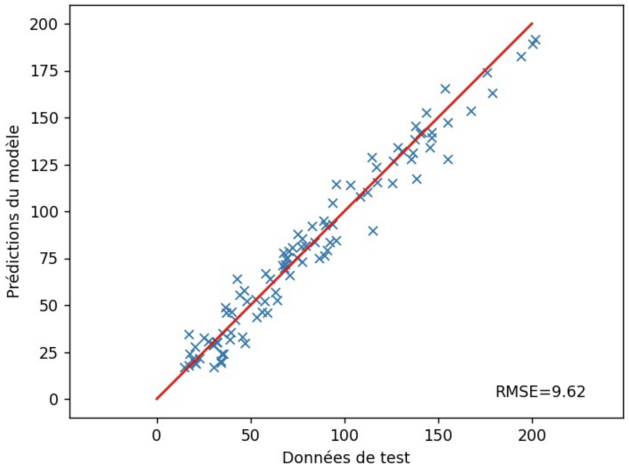


FIGURE 11 : Régression par processus gaussien sur le dataset *borehole* : inversion classique avec Cholesky complète.

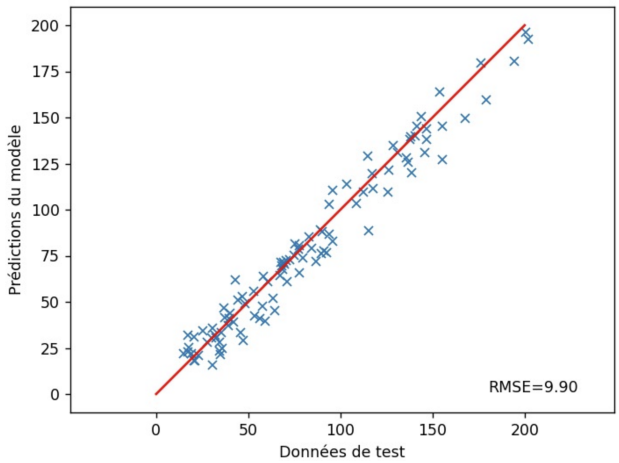


FIGURE 12 : Régression par processus gaussien sur le dataset *borehole* : inversion par Cholesky incomplète avec pivot $\eta = 0.001$.

Cette préservation de la précision se vérifie également dans des contextes multidimensionnels avec le dataset *borehole* (Figures 11 et 12), où la méthode de Cholesky incomplète continue de fournir des prédictions fiables tout en maintenant des performances de calcul nettement améliorées.

4.6. Évaluation de l'erreur par la RMSE

Afin d'évaluer quantitativement la performance de notre modèle de régression par processus gaussiens, nous calculons l'erreur quadratique moyenne (RMSE) entre les prédictions du modèle et les valeurs réelles du jeu de test. Cette mesure permet de quantifier l'écart moyen entre les prédictions $\hat{y}_i$ et les observations y_i, tout en pénalisant davantage les erreurs importantes :

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}. \tag{40}$$

Nous avons évalué deux versions de notre méthode : l'une utilisant la factorisation de Cholesky complète, et l'autre reposant sur une factorisation de Cholesky incomplète avec pivot.

— Cholesky complète : RMSE = 9,62.

— Cholesky incomplète avec pivot : RMSE = 9,90.

La version incomplète présente une légère dégradation de la précision (augmentation de l'erreur de seulement 0,28) mais offre un gain significatif en temps de calcul. Cette approche permet donc de maintenir une erreur de prédiction tout à fait raisonnable, tout en réduisant fortement la complexité computationnelle. Elle constitue ainsi un bon compromis entre efficacité et précision, particulièrement utile pour le traitement de grandes quantités de données et donc favorable à la scalabilité.

5. Vérification des résultats obtenus précédemment

5.1. Objectif de la vérification

Cette partie a pour objectif d'analyser de manière critique les résultats obtenus lors de nos premières expérimentations portant sur la décomposition de Cholesky incomplète, avec et sans pivot. Bien que la méthode avec pivot soit théoriquement conçue pour fournir une approximation de meilleure qualité, nos résultats empiriques montrent que les deux variantes produisent, dans de nombreux cas, des rangs effectifs très similaires.

Ce constat, a priori contre-intuitif, nous a conduits à mener une analyse approfondie afin d'en identifier l'origine. L'objectif n'est pas d'introduire de nouvelles méthodes, mais de vérifier la robustesse des résultats, d'en expliquer les comportements inattendus et d'en confirmer la cohérence avec le cadre théorique, en particulier celui proposé par Bach et Jordan.

5.2. Influence du choix de la norme pour la mesure de l'erreur

Nos résultats initiaux montrent que la décomposition de Cholesky incomplète avec pivot ne conduit pas systématiquement à un rang plus faible que la version sans pivot, contrairement à ce que l'on pourrait attendre par construction. Une première hypothèse naturelle a été d'attribuer ce phénomène au choix de la norme utilisée pour mesurer l'erreur d'approximation entre la matrice de covariance exacte K et son approximation $LL^\top$.

Afin de tester cette hypothèse, nous avons comparé les deux méthodes en mesurant l'erreur selon plusieurs normes matricielles, en particulier :

— la norme trace (norme L_1 matricielle),

— la norme de Frobenius (norme L_2),

qui capturent respectivement la perte de variance globale et une erreur quadratique moyenne.

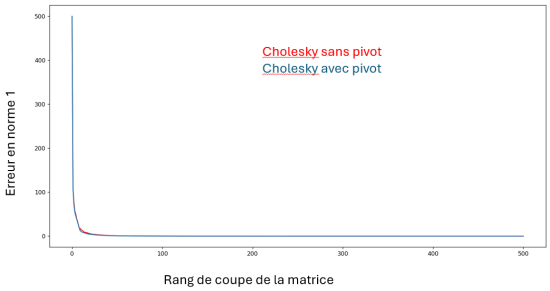


FIGURE 13 : fig : Erreur en norme 1 en fonction du rang de coupe de la matrice des matrices incomplète de cholesky avec et sans pivot

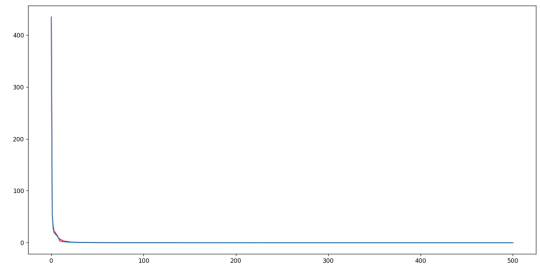


FIGURE 14 : fig : Erreur en norme 2 en fonction du rang de coupe de la matrice des matrices incomplète de cholesky avec et sans pivot

Les graphes correspondants montrent que, quelle que soit la norme considérée, l'interprétation qualitative reste inchangée : la distance entre l'approximation et la matrice exacte évolue de manière similaire pour les deux méthodes. Le choix de la norme n'influence donc pas l'ordre relatif de performance entre la décomposition avec pivot et sans pivot. Il est aussi intéressant de voir que quelque soit la norme, l'erreur d'approximation avec ou sans pivot est la même. Donc a un rang fixé, la pivot n'a pas une meilleure erreur d'approximation que sans pivot, ce qui est assez contre-intuitif. A noter que nous avons aussi essayé avec la norme infini, qui donne des résultats similaire.

5.2.1 Conclusion sur le choix de la norme

Ces résultats indiquent que le comportement observé n'est pas un artefact lié à la métrique utilisée pour quantifier l'erreur. La norme trace reste la plus pertinente dans le cadre des processus gaussiens, car elle possède une interprétation directe en termes de variance expliquée. Toutefois, le phénomène observé persiste indépendamment du choix de la norme, ce qui suggère une origine plus structurelle.

5.3. Vérification de l'implémentation par des cas particuliers

Une seconde hypothèse envisagée a été celle d'une éventuelle erreur d'implémentation de la décomposition de Cholesky incomplète avec pivot. Afin d'écarter définitivement cette possibilité, nous avons construit plusieurs cas particuliers contrôlés, pour lesquels le comportement attendu des deux méthodes est clairement identifiable.

Il est important de rappeler que la décomposition sans pivot parcourt la matrice de gauche à droite, tandis que la version avec pivot sélectionne à chaque itération la colonne maximisant l'information ajoutée, ce qui peut être interprété comme une forme de sélection gloutonne des *plus proches voisins* les plus informatifs.

5.3.1 Premier cas particulier : parcours séquentiel défavorable

Dans un premier temps, nous avons considéré une matrice de covariance dont les lignes et colonnes ont été réordonnées de manière à ce que chaque colonne soit le plus proche voisin de la précédente. Dans ce cas, l'information ajoutée à chaque étape par la méthode sans pivot est volontairement très faible, car les colonnes successives sont fortement redondantes.

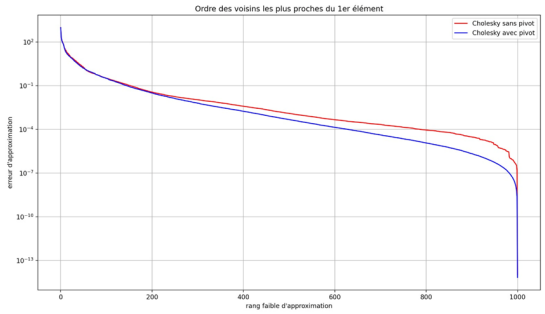


FIGURE 15 : Premier cas particulier : sélection du plus proche voisin du point précédent. La décomposition avec pivot capture plus rapidement l'information pertinente que la version sans pivot.

Comme attendu, la décomposition de Cholesky incomplète avec pivot surpasse nettement la version sans pivot, ce qui confirme la validité de l'implémentation et le rôle du pivot dans la sélection des directions informatives.

5.3.2 Deuxième cas particulier : voisinage étendu

Dans un second cas, nous avons généralisé la construction précédente en sélectionnant, à chaque étape, le plus proche voisin parmi les n points précédents, et non uniquement par rapport au dernier point ajouté. Cette configuration réduit la redondance locale tout en conservant une structure défavorable à un parcours strictement séquentiel.

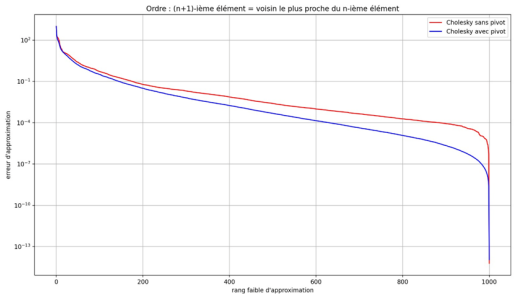


FIGURE 16 : Deuxième cas particulier : sélection du plus proche voisin parmi les n points précédents. La méthode avec pivot conserve un avantage, bien que plus modéré.

Dans ce cas également, la méthode avec pivot fournit une approximation légèrement meilleure que la méthode sans pivot, ce qui est conforme aux attentes théoriques.

5.4. Cas extrême : information concentrée en fin de matrice

Enfin, nous avons considéré un cas volontairement extrême, dans lequel toute l'information est concentrée à la fin de la matrice. Plus précisément, nous avons construit une matrice diagonale de taille 500×500, remplie de 1 sur la diagonale, à l'exception de la dernière valeur diagonale fixée à 500.

Dans cette configuration, la décomposition sans pivot ne capture la direction dominante qu'à la toute fin du parcours, tandis que la méthode avec pivot identifie immédiatement la composante principale.

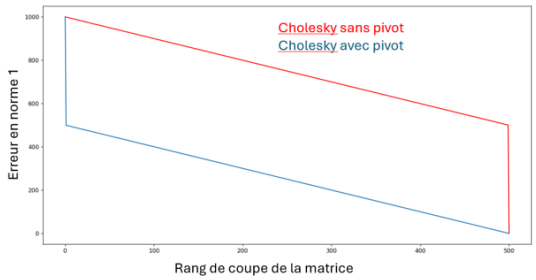


FIGURE 17 : Cas particulier extrême : information spectrale concentrée sur la dernière composante. La décomposition avec pivot capture immédiatement la direction dominante.

Ce cas illustre de manière particulièrement claire le rôle du pivot : lorsque l'information spectrale est localisée dans des directions éloignées de l'ordre naturel de parcours, le pivot devient indispensable pour obtenir une approximation efficace à faible rang.

5.5. Conclusion de la vérification

L'ensemble de ces expériences confirme que le comportement observé sur les données réelles n'est ni dû au choix de la norme, ni à une erreur d'implémentation. Il s'explique par la structure spectrale des matrices considérées. Lorsque l'information est déjà bien capturée par les premières colonnes, la décomposition sans pivot peut être aussi performante que la version avec pivot. En revanche, dans des configurations plus adverses, le pivot retrouve pleinement son intérêt.

Ces résultats valident la cohérence de nos observations avec la théorie de Bach et Jordan et soulignent que l'efficacité du pivot dépend fondamentalement de la géométrie spectrale des données.

6. NLL et hyperparamètres

6.1. Choix des hyperparamètres du noyau

Afin d'améliorer la précision des prédictions fournies par le processus gaussien, il est nécessaire d'ajuster correctement les hyperparamètres du noyau. Dans notre cas, nous utilisons un noyau gaussien (RBF), introduit précédemment, défini par :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{\|x - x'\|^2}{\ell^2}\right),$$

auquel s'ajoute un bruit gaussien indépendant de variance σ^2 sur les observations. Le modèle dépend donc de trois hyperparamètres :

$$\theta = (\sigma_f, \ell, \sigma).$$

Le choix de ces paramètres est crucial, car ils contrôlent respectivement l'amplitude des variations de la fonction cible, son degré de régularité et le niveau de bruit supposé dans les observations.

6.1.1 Estimation empirique de la variance du bruit

Nous avons dans un premier temps cherché à estimer le paramètre σ, correspondant à la variance du bruit observationnel. Deux approches sont envisageables dans le cadre d'un processus gaussien.

La première consiste à inclure directement σ parmi les hyperparamètres du modèle et à optimiser simultanément les trois paramètres (σ_f, ℓ, σ) par minimisation de la negative log-vraisemblance. Cette approche est théoriquement satisfaisante, mais elle repose sur une évaluation fiable de la vraisemblance marginale, et peut s'avérer numériquement instable ou coûteuse lorsque le bruit est mal initialisé ou lorsque la fonction objectif présente plusieurs minima locaux.

Une seconde approche consiste à estimer empiriquement la variance du bruit à partir des données, afin d'en obtenir un ordre de grandeur cohérent. En théorie, une telle estimation pourrait être réalisée à partir des résidus

$$y_i - \hat{f}(x_i),$$

où $\hat{f}$ désigne la moyenne prédictive du processus gaussien. Toutefois, cette méthode n'est pas directement applicable en pratique, puisque la fonction $\hat{f}$ elle-même dépend du processus gaussien et de ses hyperparamètres, et ne peut donc pas être évaluée indépendamment.

Pour contourner cette difficulté, nous avons adopté une estimation empirique locale du bruit, fondée uniquement sur les observations. L'idée est d'exploiter le fait que, pour des points d'entrée proches au sens de la norme euclidienne, la valeur de la fonction sous-jacente est supposée varier lentement, de sorte que la dispersion observée des sorties reflète majoritairement le bruit.

Concrètement, pour chaque point de test x_i^{test}, on considère un voisinage $\mathcal{V}_i$ constitué des points de test situés à une distance euclidienne inférieure à un seuil fixé, ou correspondant aux k plus proches voisins selon la norme ℓ_2. On calcule alors la variance empirique locale :

$$\widehat{\sigma}_i^2 = \frac{1}{|\mathcal{V}_i|} \sum_{j \in \mathcal{V}_i} \left(y_j^{\text{test}} - \overline{y}_{\mathcal{V}_i}\right)^2,$$

où $\overline{y}_{\mathcal{V}_i}$ désigne la moyenne empirique des observations dans le voisinage $\mathcal{V}_i$.

L'estimation globale de la variance du bruit est ensuite obtenue en moyennant ces variances locales sur l'ensemble des points de test :

$$\widehat{\sigma}^2 = \frac{1}{N} \sum_{i=1}^{N} \widehat{\sigma}_i^2.$$

Cette méthode permet d'obtenir une estimation robuste de l'ordre de grandeur du bruit, sans recourir à une évaluation préalable du processus gaussien. Le paramètre σ ainsi

estimé est ensuite soit fixé dans la suite des expériences, soit utilisé comme valeur initiale pour l'optimisation conjointe des hyperparamètres.

6.1.2 Vraisemblance marginale et optimisation

Les paramètres (σ_f, ℓ) sont estimés par maximisation de la vraisemblance marginale des observations.

On suppose que la fonction inconnue f suit un processus gaussien centré :

$$f \sim \mathcal{GP}(0, k),$$

où k est le noyau gaussien paramétré par (σ_f, ℓ). Les observations sont supposées bruitées selon le modèle :

$$y_i = f(x_i) + \varepsilon_i, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

où les bruits ε_i sont indépendants et identiquement distribués, et indépendants du processus f.

Sous ces hypothèses, le vecteur des observations

$$y = (y_1, \ldots, y_n)^\top$$

suit une loi normale multivariée :

$$y \sim \mathcal{N}(0, K_\sigma),$$

où

$$K_\sigma = K + \sigma^2 I,$$

et où $K \in \mathbb{R}^{n \times n}$ est la matrice de covariance définie par $K_{ij} = k(x_i, x_j)$.

La densité de probabilité associée à cette loi normale multivariée est donnée par :

$$p(y \mid X, \theta) = \frac{1}{(2\pi)^{n/2} |K_\sigma|^{1/2}} \exp\left(-\frac{1}{2} y^\top K_\sigma^{-1} y\right),$$

où $\theta = (\sigma_f, \ell, \sigma)$ désigne l'ensemble des hyperparamètres du modèle.

En prenant le logarithme de cette expression, on obtient la log-vraisemblance marginale :

$$\log p(y \mid X, \theta) = -\frac{1}{2} y^\top K_\sigma^{-1} y - \frac{1}{2} \log |K_\sigma| - \frac{n}{2} \log(2\pi).$$

Dans la pratique, il est plus commode de minimiser la *negative log-vraisemblance* (NLL), définie par :

$$\mathcal{L}(\theta) = \frac{1}{2} y^\top K_\sigma^{-1} y + \frac{1}{2} \log |K_\sigma| + \frac{n}{2} \log(2\pi).$$

6.2. Aspects numériques et calcul efficace

L'évaluation de la fonction de coût $\mathcal{L}(\theta)$ est numériquement coûteuse, car elle nécessite, à chaque itération de l'optimisation, le calcul du terme quadratique $y^\top K_\sigma^{-1} y$ ainsi que du déterminant $\log |K_\sigma|$, où

$$K_\sigma = K + \sigma^2 I.$$

Une inversion explicite de K_σ étant prohibitive pour des tailles de données importantes, nous exploitons une approximation de rang faible de la matrice de covariance K.

Plus précisément, nous effectuons une décomposition de Cholesky incomplète de K :

$$K \approx \widetilde{L}\widetilde{L}^\top,$$

où $\widetilde{L} \in \mathbb{R}^{n \times r}$ avec $r \ll n$. Cette approximation doit nous permettre de réduire fortement la complexité computationnelle.

6.3. Application de l'identité de Woodbury

Une fois l'approximation de rang faible

$$K \approx \widetilde{L}\widetilde{L}^\top$$

obtenue par décomposition de Cholesky incomplète, la matrice de covariance bruitée s'écrit :

$$K_\sigma \approx \widetilde{L}\widetilde{L}^\top + \sigma^2 I.$$

Cette structure permet d'exploiter l'identité de Woodbury afin d'éviter toute inversion de matrices de grande taille. En pratique, l'identité est utilisée pour reformuler les expressions impliquant K_σ^{-1}, mais le calcul effectif repose uniquement sur la résolution de systèmes linéaires.

Le terme quadratique apparaissant dans la negative log-vraisemblance,

$$y^\top K_\sigma^{-1} y,$$

est évalué en résolvant successivement des systèmes linéaires impliquant des matrices de taille $r \times r$, en particulier de la forme :

$$\left(I + \frac{1}{\sigma^2} \widetilde{L}^\top \widetilde{L}\right) z = \widetilde{L}^\top y,$$

sans jamais former explicitement l'inverse de cette matrice. Cette approche garantit une meilleure stabilité numérique et réduit significativement le coût computationnel.

6.4. Calcul du déterminant par la formule de Sylvester

Le calcul du terme $\log |K_\sigma|$ est réalisé à l'aide de la formule de Sylvester pour les déterminants, qui permet également d'éviter toute manipulation de matrices de grande dimension. En factorisant $\sigma^2 I$, on obtient :

$$\det(K_\sigma) = \det(\sigma^2 I) \det\left(I + \frac{1}{\sigma^2} \widetilde{L}\widetilde{L}^\top\right).$$

En appliquant la formule de Sylvester :

$$\det(I + AB) = \det(I + BA),$$

il vient :

$$\det(K_\sigma) = \sigma^{2n} \det\left(I + \frac{1}{\sigma^2} \widetilde{L}^\top \widetilde{L}\right).$$

Le déterminant de K_σ est ainsi ramené au calcul du déterminant d'une matrice symétrique définie positive de taille $r \times r$, où $r \ll n$ correspond au rang de la factorisation de Cholesky incomplète.

En pratique, ce déterminant est évalué de manière numériquement stable à l'aide d'une factorisation de Cholesky :

$$I + \frac{1}{\sigma^2}\widetilde{L}^\top \widetilde{L} = RR^\top,$$

où R est une matrice triangulaire supérieure. Le logarithme du déterminant est alors obtenu par :

$$\log\det(K_\sigma) = 2n\log\sigma + 2\sum_{i=1}^{r}\log R_{ii},$$

les termes diagonaux de R étant strictement positifs.

Cette approche permet d'éviter toute manipulation explicite de matrices de grande dimension, tout en garantissant une bonne stabilité numérique dans le calcul du terme de normalisation de la vraisemblance marginale.

6.5. Optimisation numérique

La minimisation de la negative log-vraisemblance $\mathcal{L}(\theta)$ est effectuée à l'aide de la fonction `scipy.optimize.minimize`. Plusieurs algorithmes d'optimisation ont été testés sur notre jeu de données afin d'évaluer leur capacité à minimiser efficacement la fonction objectif. En particulier, nous avons comparé les méthodes suivantes :

`L-BFGS-B`, `CG`, `SLSQP`, `BFGS`, `TNC`.

Ces algorithmes reposent sur des stratégies d'optimisation différentes (méthodes quasi-Newton, gradient conjugué, ou optimisation sous contraintes), et peuvent présenter des comportements très variables selon la structure de la fonction à minimiser et la nature des données.

Dans notre cas, la méthode `L-BFGS-B` s'est révélée être celle fournissant la meilleure minimisation de la NLL, tant en termes de valeur finale atteinte que de stabilité numérique. Ce choix est néanmoins spécifique au jeu de données considéré et à la forme particulière de la fonction de vraisemblance ; il ne constitue en aucun cas un résultat universel.

6.6. Principe de l'algorithme L-BFGS-B et paramétrisation retenue

L'algorithme `L-BFGS-B` (Limited-memory Broyden–Fletcher–Goldfarb–Shanno avec contraintes de type *Box*) est une méthode quasi-Newton destinée à l'optimisation de fonctions différentiables sous contraintes simples. Contrairement aux méthodes de Newton classiques, qui nécessitent le calcul et le stockage explicite de la matrice Hessienne, `L-BFGS-B` construit une approximation de la courbure locale à partir d'un nombre limité d'itérations précédentes, ce qui permet de réduire significativement l'empreinte mémoire tout en conservant une bonne efficacité de convergence.

Dans notre implémentation, l'optimisation n'est pas réalisée directement sur les hyperparamètres (ℓ, σ_f), mais sur leurs logarithmes :

$$\theta = (\log\ell, \log\sigma_f).$$

Cette reparamétrisation présente plusieurs avantages. Tout d'abord, elle garantit automatiquement la positivité des hyperparamètres, sans nécessiter de contraintes explicites supplémentaires. Ensuite, elle améliore le conditionnement du problème d'optimisation : les hyperparamètres du noyau peuvent varier sur plusieurs ordres de grandeur, et travailler sur leurs logarithmes permet de rendre la fonction objectif plus régulière et mieux adaptée aux méthodes quasi-Newton.

Les contraintes de type boîte de `L-BFGS-B` sont alors utilisées pour borner le domaine admissible des paramètres transformés. En particulier, nous imposons une borne inférieure sur la longueur de corrélation ℓ :

$$\ell \geq 0.8.$$

Ce choix est motivé par des considérations numériques. Lorsque ℓ devient trop petit, le noyau gaussien tend vers un noyau quasi diagonal : la matrice de covariance K se rapproche alors de la matrice identité, traduisant une quasi-indépendance entre les observations. Dans ce régime, la matrice ne présente plus de structure de rang faible exploitable, ce qui entraîne une augmentation significative du coût de la décomposition de Cholesky incomplète, dont l'efficacité repose précisément sur la présence de corrélations fortes entre les points.

Imposer une borne inférieure sur ℓ permet ainsi d'éviter des régions du domaine des hyperparamètres à la fois peu pertinentes du point de vue de la modélisation et coûteuses numériquement. Cette contrainte contribue à stabiliser l'optimisation et à réduire le temps de calcul global, tout en conservant une expressivité suffisante du noyau.

Cette combinaison (optimisation sur les log-hyperparamètres, utilisation de contraintes simples et approximation efficace de la courbure) explique le bon comportement observé de l'algorithme `L-BFGS-B` dans notre contexte expérimental.

6.7. Transition vers l'optimisation du code

L'estimation des hyperparamètres par maximisation de la vraisemblance marginale repose sur une procédure itérative, dans laquelle la fonction objectif $\mathcal{L}(\theta)$ est évaluée un grand nombre de fois. Afin d'identifier les principaux goulets d'étranglement numériques de cette procédure, nous avons réalisé des mesures de temps d'exécution détaillées pour chacune des étapes intervenant dans le calcul de la NLL.

Ces mesures montrent clairement que le coût computationnel est largement dominé par la construction de la matrice de covariance K et par le calcul de sa décomposition

de Cholesky incomplète. À l'inverse, les opérations effectuées après l'application de l'identité de Woodbury, en particulier le calcul du déterminant via la formule de Sylvester ainsi que les produits matriciels et résolutions de systèmes associés, représentent une part nettement plus faible du temps total d'exécution.

Ce constat met en évidence que l'efficacité globale de la procédure d'optimisation dépend principalement de la manière dont sont implémentées les opérations sur la matrice de covariance. Il motive ainsi naturellement le choix de concentrer les efforts d'optimisation sur ces étapes critiques, afin de rendre la méthode pleinement exploitable dans un contexte de données de grande taille.

La section suivante est donc consacrée à l'optimisation du code, avec pour objectif de réduire le coût de construction de la matrice de covariance et de la décomposition de Cholesky incomplète, tout en conservant la précision numérique et la robustesse du modèle.

7. Optimisation du code

7.1. Accélération des calculs : approche *matrix-free* et calculs associés

L'estimation des hyperparamètres par descente de gradient nécessite de réévaluer de nombreuses fois la vraisemblance marginale (ou une approximation de celle-ci). Dans ce contexte, une itération typique combine :

— la construction de la covariance K_θ,

— une factorisation de type Cholesky incomplète,

— des résolutions de systèmes linéaires pour obtenir $K_\theta^{-1} y$,

— et le calcul de termes de type log-déterminant.

Ainsi, optimiser chaque bloc de calcul devient crucial pour rendre l'apprentissage des hyperparamètres réalisable en temps raisonnable.

7.2. Woodbury : éviter l'inversion explicite et résoudre des systèmes

Dans notre cadre, on approxime la matrice de covariance K par une approximation de rang faible issue d'une Cholesky incomplète :

$$K \approx LL^\top, \qquad L \in \mathbb{R}^{n\times k}, \quad k \ll n.$$

En présence de bruit gaussien de variance σ^2, on manipule :

$$K_\sigma = K + \sigma^2 I_n \approx LL^\top + \sigma^2 I_n.$$

L'identité de Woodbury permet alors d'écrire :

$$(LL^\top + \sigma^2 I)^{-1} = \frac{1}{\sigma^2} I - \frac{1}{\sigma^4} L \left(I_k + \frac{1}{\sigma^2} L^\top L \right)^{-1} L^\top. \tag{41}$$

Une implémentation directe de (41) pourrait consister à reconstruire une matrice dense $(LL^\top + \sigma^2 I)^{-1} \in \mathbb{R}^{n\times n}$, puis à multiplier par y. Cette stratégie est cependant très coûteuse en mémoire ($\mathcal{O}(n^2)$) et en calcul.

Nous privilégions donc une approche plus efficace consistant à ne jamais former l'inverse explicitement, mais à calculer directement :

$$\alpha := (LL^\top + \sigma^2 I)^{-1} y.$$

En injectant (41), on obtient :

$$\alpha = \frac{1}{\sigma^2} y - \frac{1}{\sigma^4} L \left(I_k + \frac{1}{\sigma^2} L^\top L \right)^{-1} (L^\top y). \tag{42}$$

Le calcul se ramène donc à :

— un produit $L^\top y$ en $\mathcal{O}(nk)$,

— la formation de $M := I_k + \frac{1}{\sigma^2} L^\top L$ en $\mathcal{O}(nk^2)$,

— la résolution du système linéaire $Mz = L^\top y$ en $\mathcal{O}(k^3)$,

— puis le produit Lz en $\mathcal{O}(nk)$.

Le coût total est donc dominé par $\mathcal{O}(nk^2 + k^3)$, ce qui reste compatible avec une approche scalable tant que $k \ll n$.

7.3. Construction de K : un coût quadratique limitant

Lors des premiers tests d'optimisation des hyperparamètres, nous avons observé que le coût dominant n'était pas uniquement celui de la Cholesky incomplète, mais également celui de la construction de la matrice de covariance

$$K_{ij} = k_\theta(x_i, x_j).$$

Pour un noyau RBF en dimension d, calculer explicitement $K \in \mathbb{R}^{n\times n}$ demande typiquement :

$$\mathcal{O}(n^2 d) \text{ opérations} \qquad \text{et} \qquad \mathcal{O}(n^2) \text{ mémoire.}$$

Or, une Cholesky incomplète de rang k vise précisément à éviter un coût cubique, en se limitant à un coût proche de $\mathcal{O}(nk^2)$. Si la construction complète de K reste présente, le gain obtenu par l'approximation est en partie annulé, en particulier lorsque n devient grand.

7.4. Vectorisation du calcul du noyau

Une première amélioration consiste à vectoriser l'évaluation du noyau afin de réduire les constantes cachées. Par exemple, pour le noyau RBF isotrope :

$$k(x, x') = \tau^2 \exp\left(-\frac{\|x - x'\|^2}{2\ell^2} \right),$$

la matrice des distances au carré peut être obtenue via l'identité :

$$\|x_i - x_j\|^2 = \|x_i\|^2 + \|x_j\|^2 - 2x_i^\top x_j. \tag{43}$$

Ainsi, si $X \in \mathbb{R}^{n\times d}$ contient les observations en lignes, on peut calculer simultanément toutes les distances via des produits matriciels, ce qui évite des boucles imbriquées et accélère fortement la construction lorsque celle-ci est nécessaire.

7.5. Approche *matrix-free* : ne plus construire K, mais extraire des colonnes

L'optimisation la plus importante provient d'un constat structurel : l'algorithme de Cholesky incomplète pivotée ne requiert pas la matrice K entière, mais essentiellement :

— sa diagonale,

— et des colonnes $K_{:,i_k}$ correspondant aux pivots successifs i_k.

Rappel de l'IC pivotée. On cherche $L \in \mathbb{R}^{n\times k}$ tel que $K \simeq LL^\top$. On maintient une diagonale résiduelle $D \in \mathbb{R}^n$:

$$D_j^{(0)} = K_{jj}, \qquad D_j^{(m)} = K_{jj} - \sum_{r=1}^{m} L_{jr}^2.$$

À l'itération m, le pivot est choisi par :

$$i_m = \arg\max_j D_j^{(m-1)}.$$

La nouvelle colonne $L_{:,m}$ est définie par :

$$L_{i_m,m} = \sqrt{D_{i_m}^{(m-1)}}, \tag{44}$$

et pour tout $j \neq i_m$:

$$L_{j,m} = \frac{1}{L_{i_m,m}} \left(K_{j,i_m} - \sum_{r=1}^{m-1} L_{j,r} L_{i_m,r} \right). \tag{45}$$

Enfin, on met à jour la diagonale résiduelle :

$$D_j^{(m)} = D_j^{(m-1)} - L_{j,m}^2. \tag{46}$$

Point clé : seules les colonnes pivotées sont nécessaires. Dans (45), la seule information issue de K est le vecteur colonne

$$K_{:,i_m} = \big(k(x_1, x_{i_m}), \ldots, k(x_n, x_{i_m})\big)^\top.$$

Ainsi, il est possible d'exécuter l'algorithme sans jamais former K : à chaque itération, on calcule uniquement la colonne correspondant au pivot courant.

Extraction d'une colonne pour le noyau RBF. Pour le noyau RBF isotrope, la colonne $K_{:,i}$ est donnée par :

$$K_{j,i} = \tau^2 \exp\left(-\frac{\|x_j - x_i\|^2}{2\ell^2} \right). \tag{47}$$

En notant $x_i \in \mathbb{R}^d$ la ligne i de X, on peut calculer le vecteur des distances au carré

$$r_j^2 = \|x_j - x_i\|^2 \qquad (j = 1, \ldots, n)$$

en coût $\mathcal{O}(nd)$, puis appliquer l'exponentielle composante par composante pour obtenir la colonne en $\mathcal{O}(n)$.

Coût global. Si l'algorithme s'arrête au rang k, l'approche *matrix-free* nécessite :

$$k \text{ colonnes extraites} \quad \Rightarrow \quad \mathcal{O}(knd)$$

pour l'évaluation du noyau, au lieu de $\mathcal{O}(n^2 d)$ pour la construction complète de K. La mémoire est également réduite, car on ne stocke que :

$$L \in \mathbb{R}^{n\times k} \quad \text{et quelques vecteurs auxiliaires,}$$

soit $\mathcal{O}(nk)$ au lieu de $\mathcal{O}(n^2)$.

Compatibilité avec les produits matriciels. Cette approche est particulièrement adaptée au cas pivoté car l'algorithme effectue naturellement des opérations de type :

$$L[J, : m]\, L[i_m, : m]^\top,$$

c'est-à-dire des produits impliquant uniquement L (déjà stocké) et la colonne courante $K_{:,i_m}$ calculée à la volée. Ainsi, la matrice K n'est jamais requise sous forme dense, et l'algorithme reste entièrement exploitable dans un cadre *scalable*.

7.6. Conclusion

Ces optimisations ont permis de réduire drastiquement le coût d'une itération d'apprentissage des hyperparamètres. En particulier, l'approche *matrix-free* supprime la construction explicite de K et transforme un coût en $\mathcal{O}(n^2 d)$ en un coût en $\mathcal{O}(knd)$, cohérent avec l'objectif d'une approximation de rang faible. Combinée à l'utilisation de Woodbury sous forme de résolutions de systèmes linéaires, elle rend l'optimisation itérative des hyperparamètres nettement plus viable sur des jeux de données de grande taille.

8. Résultats - suite

8.1. Dataset "bouquetins"

Nous avons désormais accès à des données correspondant au problème pratique de la propagation de la brucellose chez les bouquetins. Dans ces données, les points d'applications de la fonction à prédire ont trois coordonnées pouvant chacune prendre 6 valeurs différentes pour un total de 216 scénarios possibles. Sur chacun de ces 216 scénarios, 1000 mesures ont été effectuées ; soit un total de 216 000 lignes que nous mélangeons pour nous assurer, comme précédemment, de choisir nos lignes d'entraînement et de test au hasard parmi la base de données.

8.2. Hyperparamètres

Changer de base de données signifie également mettre à jour les hyperparamètres.

Paramètre σ : σ modélise la variation typique des données. Nous donc associons σ à la moyenne des écart-types des données du dataset, soit 14.36.

Paramètres l et τ : Comme expliqué précédemment, les valeurs optimales des hyperparamètres l et τ sont calculées via une descente de gradient. Nous générons une heatmap représentant la Negative-Log-Likelyhood en fonction de ces paramètres. Le nombre d'observations est fixé ici à 1000.

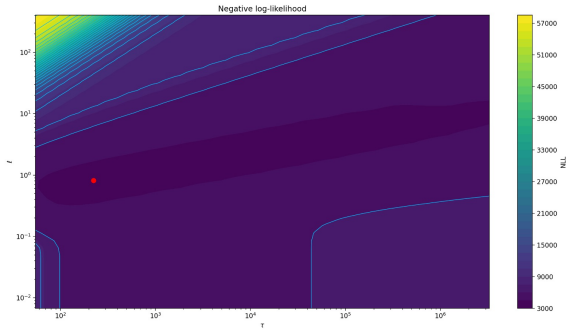


FIGURE 18 : Log-vraisemblance négative

Le point rouge représente le couple d'hyperparamètres minimisant la NLL, soit $l = 0.8145$ et $\tau = 207.1272$.

Ces hyperparamètres seront utilisés dans la suite des applications.

8.3. Performance temporelle : Cholesky *matrix-free*

De la même manière que nous avons comparé la performance temporelle des décompositions de Cholesky complète et incomplète avec/sans pivot, cette section vise à valoriser le gain de temps conféré par l'approche *matrix-free* établie plus tôt. Nous mesurons et comparons à une précision $\eta = 10^{-3}$ fixe, pour différents nombre de lignes de dataset $n \in [600, 2000]$ (avec une incrémentation de 100) deux temps moyens sur 3 exécutions :

— D'une part, la performance temporelle de la décomposition Cholesky incomplète avec pivot **additionnée au temps de construction de la matrice de covariance de nos données d'entraînements**.

— D'autre part, la performance temporelle de la décomposition Cholesky incomplète avec pivot avec extraction de colonnes **sans y ajouter le temps de construction de la matrice de covariance**.

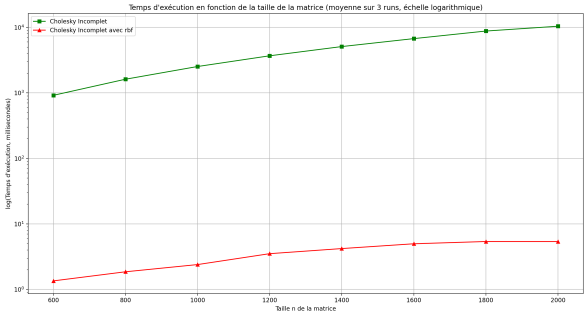


FIGURE 19 : Comparaison des performances temporelles de la décomposition de Cholesky incomplète avec pivot et sa variante matrix-free

Ce résultat est important car il montre à quel point la variante matrix-free de la décomposition élimine une grosse partie du temps de calcul de la matrice de covariance comme constaté sur cette figure.

8.4. Application du processus gaussien optimisé au dataset des bouquetins

Pour 10000 points d'observations, 1000 points de test, $l = 0.8145$, $\tau = 207.1272$ et $\sigma = 14.36$, nous obtenons les résultats suivants avec une RMSE égale à 14.8478 très proche de l'écart-type moyen σ.

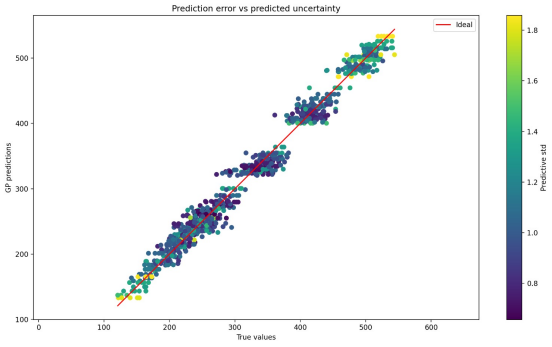


FIGURE 20 : Valeurs attendues vs. Prédictions du GP

9. Conclusion

9.1. Synthèse des résultats

L'ensemble des expérimentations menées confirme l'intérêt majeur de la décomposition de Cholesky incomplète, notamment avec pivot, pour améliorer la scalabilité des processus gaussiens sans compromettre leur précision. D'un point de vue computationnel, les analyses montrent une réduction très nette du temps d'exécution par rapport à la méthode de Cholesky complète, en particulier lorsque la taille de la matrice de covariance augmente. Cette amélioration est directement liée à la structure spectrale des matrices issues de noyaux lisses, dont la décroissance

rapide des valeurs propres permet une approximation efficace à faible rang.

En parallèle, les comparaisons effectuées sur des tâches de régression unidimensionnelle mettent en évidence que la méthode approximative conserve des performances prédictives quasiment identiques à celles de l'approche exacte, tant en moyenne qu'en variance prédictive. Cette stabilité s'observe également dans des cas multidimensionnels, ce qui renforce la robustesse de la méthode. Le gain en complexité s'accompagne donc d'une préservation fiable des propriétés probabilistes du modèle, ce qui en fait une solution particulièrement adaptée pour l'émulation de simulateurs coûteux ou de grande taille.

Suite à un raffinement de nos algorithmes, le choix d'hyperparamètres est désormais automatique et la complexité temporelle est davantage réduite. D'autre part, suite à un traitement des données brutes concernant le troupeau de bouquetins, le processus gaussien a pu y être appliqué. Les résultats et la précision exhibée sont satisfaisants.

9.2. Perspectives

Malgré les avancées faites sur notre algorithme, il reste encore à étudier l'impact qu'aurait l'utilisation de la décomposition de Cholesky avec Side Information discutée dans [1] sur notre régression par processus gaussien.

Ensuite, il va sans dire que les bases de données sur lesquelles nous avons fait tourner notre algorithme sont très particulières. Le dataset "borehole" exhibe des propriétés spectrales atypiques qui rendent l'utilisation du pivot caduque à partir d'un certain nombre d'observations, et le dataset des bouquetins de contient que 216 scénarios possibles. Ainsi il serait intéressant d'appliquer le processus gaussien à d'autres datasets.

Références

[1] Francis R. Bach and Michael I. Jordan. Predictive low-rank decomposition for kernel methods. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 33–40, Bonn, Germany, 2005. ACM.

[2] Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins Studies in the Mathematical Sciences. The Johns Hopkins University Press, Baltimore, Md., 4th edition, 2013.

[3] Haitao Liu, Yew-Soon Ong, Xiaobo Shen, and Jianfei Cai. When gaussian process meets big data : A review of scalable gps. *IEEE Transactions on Neural Networks and Learning Systems*, 31(11) :4405–4423, 2020.

[4] Nordine Rajaoui. Comprendre le processus gaussien : une approche probabiliste pour la régression, 2023.

[5] Thomas Romary. Incomplete cholesky decomposition for the kriging of large datasets. *Spatial Statistics*, 5 :85–99, 2013.

[6] Joe Suzuki. *Kernel Methods for Machine Learning with Math and Python : 100 Exercises for Building Logic*. Springer, 2022.